

SI1001 Teoría de la Computación

Verificación de programas

Andrés Sicard Ramírez

Universidad EAFIT

Semestre 2025-2

Verificación de programas

Preliminares

- El texto guía para estas diapositivas es (Grassman y Tremblay 1996, cap. 9).
- Los números asignados a las definiciones, ejemplos, ejercicios, figuras, páginas, secciones y teoremas en estas diapositivas corresponden a los números asignados en el texto guía.

Cinco preguntas

En el prefacio de (Hein 2017) el autor señala que la mayoría de los problemas en Ciencias de la Computación involucran una de las siguientes cinco preguntas:

- (i) *«Can the problem be solved by a computer program?»*
- (ii) *«If not, can you modify the problem so that it can be solved by a program?»*
- (iii) *«If so, can you write a program to solve the problem?»*
- (iv) *«Can you convince another person that your program is correct?»*
- (v) *«Can you convince another person that your program is efficient?»*

Antecedentes

- «*Checking a Large Routine*» (Turing, 1949)
- «*A Basis for a Mathematical Theory of Computation*» (McCarthy, 1963)
- «*Proof of Algorithms by General Snapshots*» (Naur, 1966)
- «*A Constructive Approach to the Problem of Program Correctness*» (Dijkstra, 1967)
- «*Assigning Meanings to Programs*» (Floyd, 1967)
- «*An Axiomatic Basis for Computer Programming*» (Hoare, 1969)
- LCF (*Logic for Computable Functions*) (Scott, 1969, 1993)
- «*Procedures and Parameters: An Axiomatic Approach*» (Hoare, 1971)

Clasificación (paradigmas) de los lenguajes de programación

Clasificación

- Declarativos (¿qué?)
 - Funcionales (LISP, ML, HASKELL, ...)
 - Lógicos, basados en restricciones (PROLOG, CLP, ...)
- Imperativos (¿cómo?)
 - von Neumann (FORTRAN, PASCAL, C, ADA, ...)
 - Orientados a objetos (SMALLTALK, C++, JAVA, ...)
 - *Scripting* (PERL, PYTHON, PHP, ...)

Clasificación (paradigmas) de los lenguajes de programación

Clasificación^a

- Declarativos (¿qué?)
 - Funcionales (LISP, ML, HASKELL, ...)
 - Lógicos, basados en restricciones (PROLOG, CLP, ...)
- Imperativos (¿cómo?)
 - **von Neumann** (FORTRAN, **PASCAL**, **C**, ADA, ...)
 - Orientados a objetos (SMALLTALK, C++, JAVA, ...)
 - *Scripting* (PERL, PYTHON, PHP, ...)

^aClasificación adaptada de la clasificación presentada en (Scott 2015).

La arquitectura de von Neumann

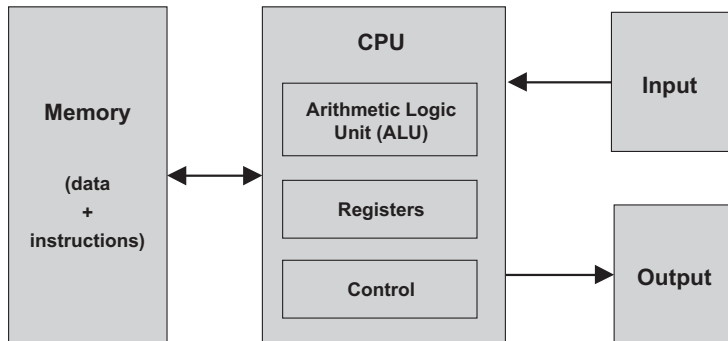


Figura 5.1 en (Nisan y Shimon 2005).

Programas en lenguajes de máquina

Ejemplo

Programa en lenguaje de máquina que adiciona los números 5 y 6.

```
0010001000000100
0010010000000100
0001011001000010
0011011000000011
1111000000100101
0000000000000101
0000000000000110
```

Figura 1.2 en (Louden y Lambert 2011).

Programas en lenguajes ensambladores

Ejemplo

Programa en lenguaje ensamblador que adiciona los números en las variables `FIRST` y `SECOND` y almacena el resultado en la variable `SUM`.

```
.ORIG x3000      ; Address (in hexadecimal) of the first instruction
LD  R1, FIRST    ; Copy the number in memory location FIRST to register R1
LD  R2, SECOND    ; Copy the number in memory location SECOND to register R2
ADD R3, R2, R1    ; Add the numbers in R1 and R2 and place the sum in
                  ; register R3
ST  R3, SUM       ; Copy the number in R3 to memory location SUM
HALT              ; Halt the program
FIRST  .FILL #5   ; Location FIRST contains decimal 5
SECOND .FILL #6   ; Location SECOND contains decimal 6
SUM     .BLKW #1   ; Location SUM (contains 0 by default)
.END            ; End of program
```

Figura 1.3 en (Louden y Lambert 2011).

Programas en lenguajes imperativos

Ejemplo

Programa en lenguaje C que adiciona los números en las variables `first` y `second` y almacena el resultado en la variable `sum`.

```
int first = 5;  
int second = 6;  
sum = first + second;
```

Programas en lenguajes imperativos

Descripción

Un programa en un lenguaje imperativo es una secuencia de instrucciones que representan comandos:

- instrucciones de asignación,
- instrucciones de secuenciación,
- instrucciones *if-then-else*,
- instrucciones *while*.

Razonamiento sobre programas

Categorías

La investigación acerca el razonamiento sobre programas se puede dividir en cuatro categorías (Achten et al. 2010):

- (i) Definir la semántica (nuevos conceptos).
- (ii) Razonamiento sobre transformaciones de programas.
- (iii) **Verificación (formal) de propiedades funcionales** (correspondencia de entrada y salida).
- (iv) Verificación (formal) de propiedades no funcionales (consumo de memoria, rendimiento paralelo, etc.).

Correctitud de programas

Correctitud parcial y correctitud total

Al establecer la correctitud funcional (correspondencia de entrada y salida) de un programa se distingue entre:

- **Correctitud parcial:** El programa es correcto respecto a su especificación.
- **Correctitud total:** Correctitud parcial + terminación.

Algunos sistemas (herramientas) para la verificación de programas

Para lenguajes específicos

- CakeML (implementación verificada de ML).
- CompCert (implementación verificada de un subconjunto de C).
- Frama-C y su plugin WP (programas en C).
- KeY (programas en JAVA)

Algunos sistemas (herramientas) para la verificación de programas

Para lenguajes específicos

- CakeML (implementación verificada de ML).
- CompCert (implementación verificada de un subconjunto de C).
- Frama-C y su plugin WP (programas en C).
- KeY (programas en JAVA)

Sistemas generales

- Agda.
- Idris.
- Isabelle/HOL.
- HOL4, HOL Light.
- Rcoq (antes Coq).
- Why3.

Lógica de Hoare: Conceptos preliminares

Descripción

Emplearemos la lógica de Hoare para verificar programas de un subconjunto del lenguaje imperativo PASCAL:

- (i) operadores aritméticos,
- (ii) funciones de la biblioteca estándar,
- (iii) bloques **begin-end** ,
- (iv) instrucciones de asignación (**:=**),
- (v) instrucciones de secuenciación (**;**),
- (vi) instrucciones **if-then** y **if-then-else** ,
- (vii) instrucciones **while** .

Lógica de Hoare: Conceptos preliminares

Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

Lógica de Hoare: Conceptos preliminares

Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

Descripción

Las **sentencias (proposiciones)** de la lógica de Hoare son sentencias (proposiciones) de la lógica de predicados de primer orden en las variables del programa.

Lógica de Hoare: Conceptos preliminares

Descripción

El **estado de un programa** consiste del valor de la variables utilizadas por el programa.

Descripción

Las **sentencias (proposiciones)** de la lógica de Hoare son sentencias (proposiciones) de la lógica de predicados de primer orden en las variables del programa.

Notación

Para escribir las sentencias emplearemos las siguientes constantes lógicas: $\wedge$ (conjunción), $\vee$ (disyunción), $\Rightarrow$ (implicación), $\neg$ (negación) y $=$ (igualdad).

Lógica de Hoare: Conceptos preliminares

Definición 9.1

Una **aserción** (**afirmación**) es una sentencia referente a un estado de programa.

Lógica de Hoare: Conceptos preliminares

Definición 9.1

Una **aserción** (**afirmación**) es una sentencia referente a un estado de programa.

Notación

Las aserciones se escribirán entre llaves. Es decir, $\{A\}$ denota que A es una aserción.

Lógica de Hoare: Conceptos preliminares

Definición 9.2

«Si C es una parte de un código, entonces cualquier aserción $\{P\}$ se denomina **precondición** de C si $\{P\}$ sólo implica el estado inicial. Cualquier aserción $\{Q\}$ se denomina **postcondición** [de C] si $\{Q\}$ sólo implica el estado final. Si C tiene como precondición a $\{P\}$ y como postcondición a $\{Q\}$, se escribe $\{P\} C \{Q\}$. La terna $\{P\} C \{Q\}$ se denomina **terna de Hoare**.»

Lógica de Hoare: Conceptos preliminares

Ejemplo (terna de Hoare)

Para la terna de Hoare

$$\{y \neq 0\} \ x := 1/y \ \{x = 1/y\},$$

- la precondition $y \neq 0$ afirma que el valor de y es diferente de 0,
- la postcondition $x = 1/y$ afirma que el valor de x es $1/y$,
- y el programa con la precondition y postcondition anteriores es la instrucción $x := 1/y$.

Lógica de Hoare: Conceptos preliminares

Ejemplo (precondición vacía)

La terna de Hoare

$$\{\} \text{ a } := \text{ b } \{a = b\},$$

tiene una precondición vacía $\{\}$ la cual se interpreta como «verdadera para todos los estados del programa».

Lógica de Hoare: Conceptos preliminares

Pregunta

La instrucción `h := a; a := b; b := a` intercambia los valores de las variables `a` y `b`.
¿Cómo escribir una postcondición para esta instrucción?

Lógica de Hoare: Conceptos preliminares

Métodos para establecer la distinción entre los valores de las variables en el estado inicial y en el estado final

(i) Uso de subíndices

El subíndice α se empleará para los valores iniciales de las variables y el subíndice ω se empleará para los valores finales de las variables.

(ii) Uso de variables ocultas

Se utilizan variables que no aparecen en el código para almacenar los valores iniciales de las variables.

Lógica de Hoare: Conceptos preliminares

Ejemplos (diferenciando el estado inicial y el estado final)

Ternas de Hoare para la instrucción que intercambia los valores de dos variables.

1. Empleando los subíndices α y ω

$$\{\} \text{ h := a; a := b; b := a } \{(a_\omega = b_\alpha) \wedge (b_\omega = a_\alpha)\}.$$

2. Empleando las variables ocultas A y B

$$\{a = A, b = B\} \text{ h := a; a := b; b := a } \{a = B, b = A\}.$$

Lógica de Hoare: Conceptos preliminares

Ejemplo

Una terna de Hoare para una instrucción **if-then-else**.

$$\{ \text{if } x > y \text{ then } m := x \text{ else } m := y \\ \{ (x > y \Rightarrow m = x_\alpha) \wedge (\neg(x > y) \Rightarrow m = y_\alpha) \} \}.$$

Lógica de Hoare: Conceptos preliminares

Definición 9.3

«Si C es un código con la precondition $\{P\}$ y la postcondición $\{Q\}$, entonces $\{P\} C \{Q\}$ se dice que es **parcialmente correcto** si el estado final de C satisface $\{Q\}$, siempre que el estado inicial satisfaga $\{P\}$. [El programa] C también sería parcialmente correcto si no hay estado final debido a que el programa no termina. Si $\{P\} C \{Q\}$ es parcialmente correcto y C termina, entonces se dice que $\{P\} C \{Q\}$ es **totalmente correcto**.»

Referencias

-  Peter Achten, Marko van Eekelen, Pieter Koopam y Marco T. Morazán (2010). Trends in *Trends in Functional Programming* 1999/2000 versus 2007/2008. Higher-Order Symbolic Computation 23.4, págs. 465-487. DOI: [10.1007/s10990-011-9074-z](https://doi.org/10.1007/s10990-011-9074-z) (vid. pág. 12).
-  Robert W. Floyd (1967). Assigning Meanings to Programs. En: Mathematical Aspects of Computer Science. Ed. por Jacob T. Schwartz. Vol. 19. Proceedings of Symposia in Applied Mathematics. 1967, págs. 19-32 (vid. pág. 4).
-  Karl Grassman y Jean-Paul Tremblay (1996). Matemáticas Discretas y Lógica. Una Perspectiva desde la Ciencia de la Computación. Trad. por Rafael García-Bermejo, María Luisa Díez Platas y Vivian de los Ángeles Fernández Vásquez. Prentice Hall, 1996 (vid. pág. 2).
-  James L. Hein (2017). Discrete Structures, Logic, and Computability. 4.^a ed. Jones & Bartlett Learning, 2017 (1995) (vid. pág. 3).
-  C. A. R. Hoare (1969). An Axiomatic Basis for Computer Programming. Communications of the ACM 12.10, 576-580(3). DOI: [10.1145/363235.363259](https://doi.org/10.1145/363235.363259) (vid. pág. 4).

Referencias

-  C. A. R. Hoare (1971). Procedures and Parameters: An Axiomatic Approach. En: Symposium on Semantics of Algorithmic Languages. Ed. por E. Engler. Vol. 188. Lecture Notes in Mathematics. Springer Berlin Heidelberg, 1971, págs. 102-116. DOI: [10.1007/BFb0059696](https://doi.org/10.1007/BFb0059696) (vid. pág. 4).
-  Kenneth C. Loudon y Kenneth A. Lambert (2011). Programming Languages. Principles and Practice. 3.^a ed. Cengage Learning, 2011 (1993) (vid. págs. 8, 9).
-  John McCarthy (1963). A Basis for a Mathematical Theory of Computation. En: Computer Programming and Formal Systems. Ed. por P. Braffort y D. Hirshberg. North-Holland, 1963, págs. 33-70 (vid. pág. 4).
-  Peter Naur (1966). Proof of Algorithms by General Snapshots. BIT 6.4, págs. 310-316 (vid. pág. 4).
-  Noam Nisan y Schocken Shimon (2005). The Elements of Computing Systems. Building a Modern Computer from First Principles. MIT Press, 2005 (vid. pág. 7).
-  Michael L. Scott (2015). Programming Language Pragmatics. 4.^a ed. Morgan Kaufmann, 2015 (1999) (vid. pág. 6).

Referencias



Alan Turing (1949). Checking a Large Routine. En: Report of a Conference on High Speed Automatic Calculating Machines. 1949, págs. 67-69 (vid. [pág. 4](#)).